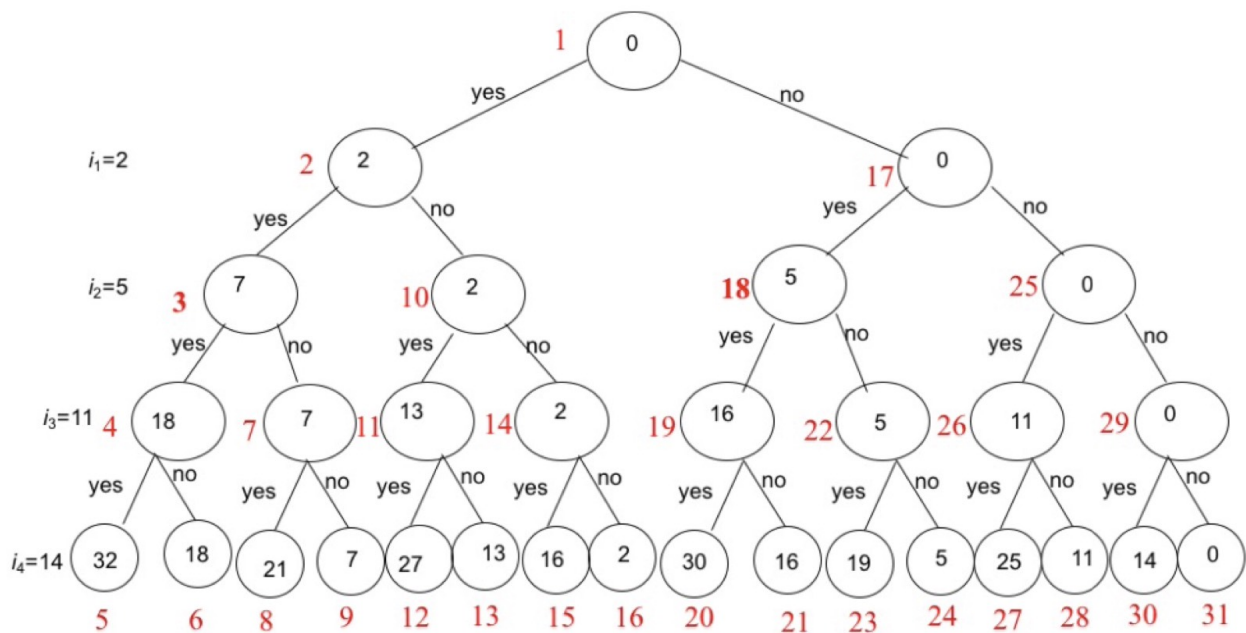


Design and Analysis of Algorithms
CS575, Spring 2026
Theory Assignment 4

1. [20%] A set of integers $A = \{2, 5, 11, 14\}$ is given. For the given set A , find **every subset** that sums to $S = 16$.
- [10%] Solve the problem using the depth first method. Draw a state space tree and clearly show every step. Also, number the nodes in the sequence of visiting them.
 - [10%] Find the subsets via backtracking. Draw a (pruned) state space tree and clearly show every step. Number the nodes in the sequence of visiting them too.

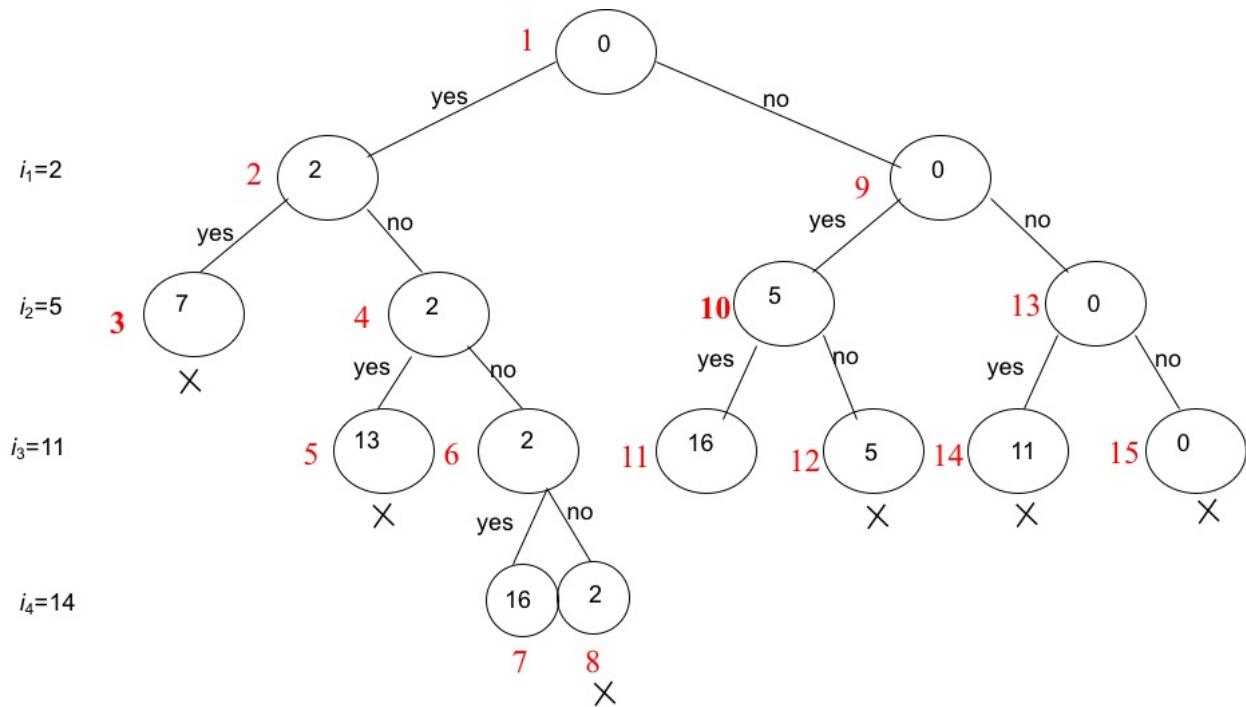
Answer:

- a. There are two solutions: $\{2, 14\}$ and $\{5, 11\}$.



Only the leaf nodes 15 and 21 satisfy that the sum of subset is equal to 16.

- b. The weights are already sorted. Otherwise, you need to sort the weights (range 1 through $n=4$).



You need to show it for each node.

e.g. Node 4: weight = 2, weightSoFar = 2, totalPossibleLeft = 25. The Node is promising because $\text{weightSoFar} + \text{totalPossibleLeft} = 27 \geq 16$ and $\text{weightSoFar} + w[3] = 2 + 11 = 13 \leq 16$.

Node 5: weight = 13. The node is non-promising because $\text{weightSoFar} + w[4] = 27 > 16$.

Node 8: weight = 2. The node is non-promising because $\text{weightSoFar} + \text{totalPossibleLeft} = 2 + 0 = 2 < 16$.

Node 15: weight = 0. The node is non-promising because $\text{weightSoFar} + \text{totalPossibleLeft} = 14 < 16$.

2. [10%] Consider the following revised sum of subsets problem: **Given n positive integers $w_1, \dots, w_n$ and two positive integers S_1 and S_2 , find all subsets of $w_1, \dots, w_n$ that add to a number between S_1 and S_2 (including S_1 and S_2).** Suppose we follow the same general backtracking method for solving the original sum of the subsets problem (see the slides of the Backtracking chapter) and use the same notations for weightSoFar and totalPossibleLeft. Define the condition(s) for a node in the state space tree to be promising.

Answer: A node is promising if *both* of the following are true:

- $\text{weightSoFar} + \text{totalPossibleLeft} \geq S_1$
- $\text{weightSoFar} + w_{i+1} \leq S_2$ or $S_1 \leq \text{weightSoFar} \leq S_2$

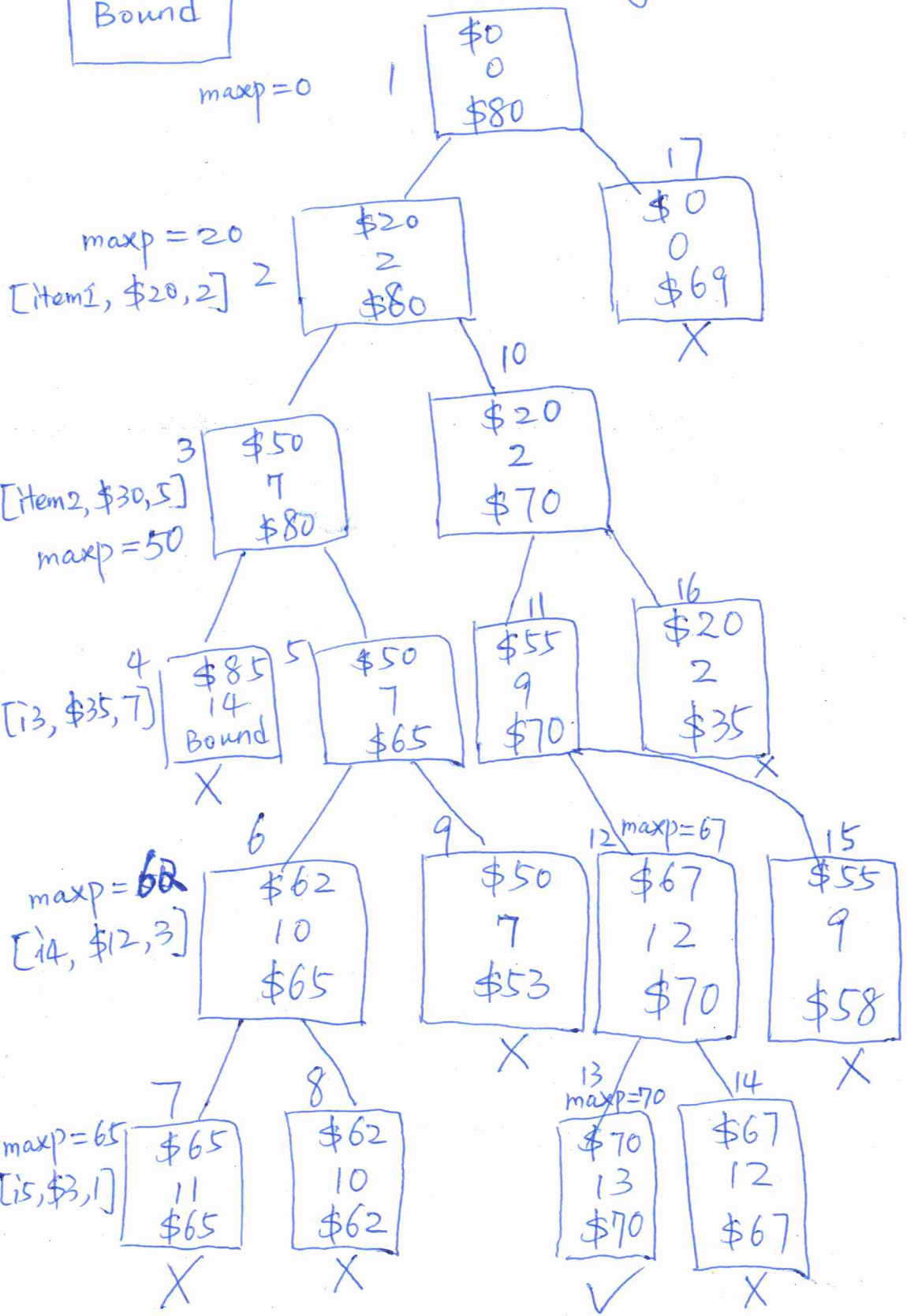
3. [30%] When the capacity of the knapsack is 13, solve the following 0/1-knapsack problem using (a) [10%] the backtracking algorithm; (b) [10%] the Breadth-First search with branch and bound pruning algorithm and (c) [10%] the Best-first search with branch and bound pruning algorithm in Chapter 13. All algorithms use the optimal fractional knapsack algorithm to compute the upper bound of the profit. Show the actions step by step.

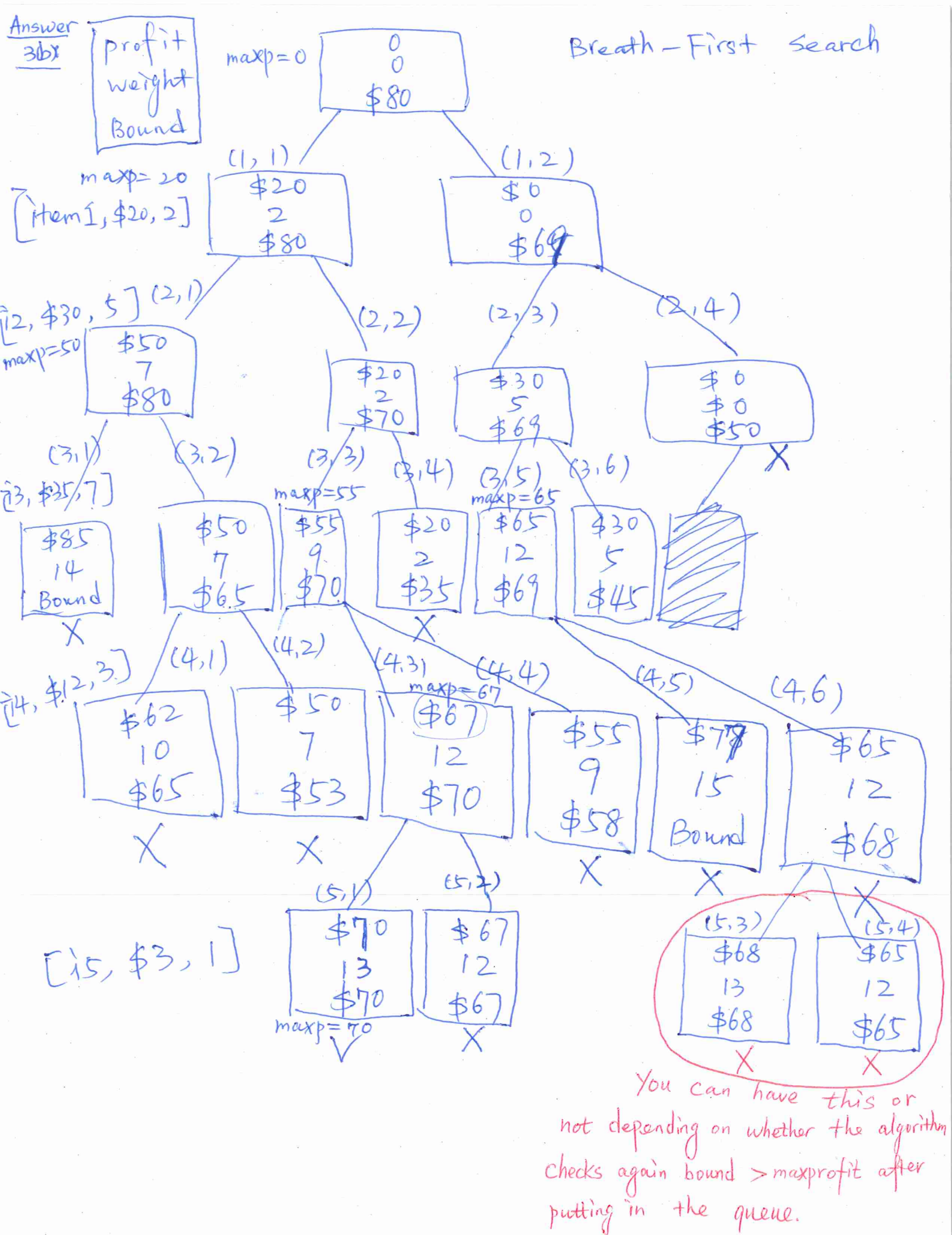
i	p_i	w_i	p_i / w_i
1	\$20	2	\$10
2	\$30	5	\$6
3	\$35	7	\$5
4	\$12	3	\$4
5	\$3	1	\$3

Answer: 3(a)

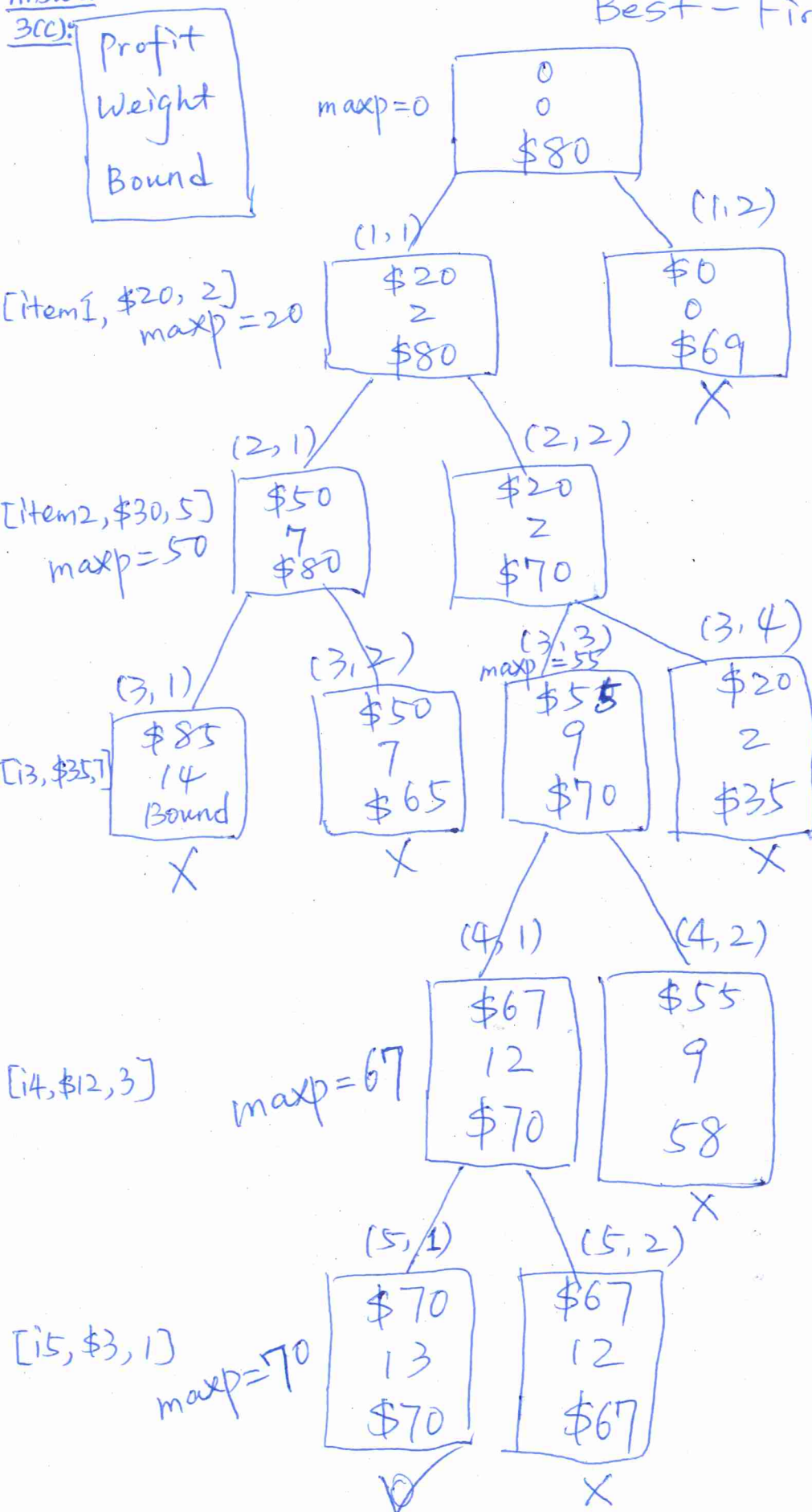
Profit
Weight
Bound

Depth-First Search Backtracking Algorithm.





Answer 3(c): Best-First search.



4. [20%] When the capacity of the knapsack is 15, solve the following **0-1 knapsack** problem using the dynamic programming approach. (a) [10%] Please show B[n, W] table; (b) [10%] please show what items you should include in the knapsack to derive the maximum benefit and how you derive the items that should be included in the knapsack.

i	p_i	w_i	p_i / w_i
1	\$20	2	\$10
2	\$30	5	\$6
3	\$35	7	\$5
4	\$12	3	\$4
5	\$3	1	\$3

a.

	w=0	w=1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
i=0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
i=1	0	0	20	20	20	20	20	20	20	20	20	20	20	20	20	20
2	0	0	20	20	20	30	30	50	50	50	50	50	50	50	50	50
3	0	0	20	20	20	30	30	50	50	55	55	55	65	65	85	85
4	0	0	20	20	20	32	32	50	50	55	62	62	67	67	85	85
5	0	3	20	23	23	32	35	50	53	55	62	65	67	70	85	88

b. {i1, i2, i3, i5} The process are labeled in yellow.

1. Consider a directed weighted graph $G(V, E)$, where V is the set of vertices and E is the set of edges. Suppose we use adjacency list to implement the graph. Consider a sequence of $|V|$ operations with each corresponding to a different vertex in V . For the operation for vertex v , the cost is assumed to be the number of outgoing edges from v .

- (a) What is the cost of the worst possible single operation in this sequence of operations?

Answer: $|E|$. This happens when there is a vertex that has $|E|$ outgoing edges while other vertices don't have any outgoing edges.

- (b) What is the average amortized cost of the above sequence of operations based on aggregate analysis?

Answer: Since the total number of outgoing edges of G is $|E|$, the total number of the edges that will be involved in the sequence of $|V|$ operations will be $|E|$. So the average amortized cost of this sequence of operations is $|E| / |V|$.

2. Consider **Problem A**: Given an undirected graph $G = (V, E)$, determine whether G is a complete graph.

- a. Does problem A belong to the class P? If no, explain why; if yes, describe and analyze a polynomial bound algorithm (in terms of $|V|$) that solves it. (Hint: check there are no missing edges).

Answer: Yes. The following algorithm can answer the question correctly:

```

for each vertex  $u$  in  $V$ 
    for each vertex  $v$  in  $V$ 
        if  $(u, v) \notin E$  return "No" // not a complete graph
return "Yes" // no missing edge, the graph is a complete graph

```

The above algorithm has $O(|V|^2)$ time (note that the third line takes constant time if the graph is implemented by an adjacency matrix). Since the algorithm has a polynomial-time upper bound, it is in P.

- b. Does problem A belong to the class NP? Explain.

Answer: Yes, because $P \subset NP$ or because any solution that can be obtained in polynomial time can be verified in polynomial time.

3. Consider the following attempt to transform the Hamiltonian Cycle problem (HCP) to the Traveling Salesman problem (TSP). Let $G = (V, E)$ be an instance of HCP. We construct an instance (G', d, B) of TSP as follows:

- a. $G' = (V', E')$ is a graph with
 - i. $V' = V$, i.e., no change to the set of vertices

- ii. $E' = \{(u, v) \mid u, v \in V', u \neq v\}$, i.e., form an edge between each pair of vertices in V' , making G' a complete graph.
- b. $d(\cdot)$ is the distance function for each edge in E' defined below:

$$d(u, v) = \begin{cases} 1 & \text{if } (u, v) \in E \\ 2 & \text{if } (u, v) \notin E \end{cases}$$

Now answer the following questions about this transformation:

- a. Find an appropriate value b for bound B for the TSP and show that for this bound G has a Hamiltonian cycle if and only if G' has a tour with a total distance at most b .

Answer: An appropriate value for b is $|V|$. Note that a Hamiltonian cycle has exactly $|V|$ edges (a tree has $|V| - 1$ edges but it does not form a cycle). If G has a Hamiltonian cycle, the cycle must have exactly $|V|$ edges. Since each such an edge has a distance of 1, the total distance of the cycle is $b = |V|$. Thus this cycle is a tour of G' with a total distance at most b . On the other hand, if G' has a tour with a total distance at most $b = |V|$, the tour must have $|V|$ edges (because it is a cycle). Since the distance on each edge in G' can be only 1 or 2, it means that all edges in this tour must have distance 1. Thus, the edges in this tour are all in G , meaning that the tour is a Hamiltonian cycle of G .

- b. Show that the above transformation takes polynomial time in terms of $|V|$.

Answer: Forming V' takes time $O(|V|)$. Forming E' takes at most time $O(|V|^2)$ (the complete graph with $|V|$ vertices has $|V| * (|V| - 1)$ edges). Assigning the distances to all edges takes at most time $O(|V|^2)$. Thus, the transformation takes time at most $O(|V|^2)$, which is polynomial.